CPE361 Pinku Clinic · ระบบจองคิวคลินิก — ขั้นที่ 1 ออกแบบฐานข้อมูล

วิชา CPE361 · ปรับปรุงล่าสุด 1 พ.ย. 2026 · Nutnicha K.

1. ภาพรวม 6 ตาราง

ระบบเก็บข้อมูลใน 6 ตาราง: 3 ตารางเป็นข้อมูลหลักที่อยู่ถาวร (แผนก แพทย์ ผู้ป่วย) 1 ตารางเป็นตารางออกตรวจของแพทย์ 1 ตารางเป็นประวัติการจองคิว และ 1 ตารางเป็นผลการรักษาของคิวที่ตรวจแล้ว

ตาราง	ชื่อไทย	เก็บอะไร	1 แถว คือ
departments	แผนก	ชื่อแผนกของคลินิก	1 แผนก
doctors	แพทย์	ชื่อแพทย์ และแผนกที่สังกัด	แพทย์ 1 คน
patients	ผู้ป่วย	ชื่อ เบอร์โทร วันเกิด	ผู้ป่วย 1 คน
schedules	ช่องเวลาออกตรวจ	วันและเวลาที่แพทย์ออกตรวจ ช่องละ 30 นาที สร้างไว้ล่วงหน้า 30 วัน	1 ช่องเวลาของแพทย์ 1 คน
appointments	การจองคิว	ผู้ป่วยคนไหนจองช่องเวลาไหน อาการ สถานะ เวลาที่จอง	การจอง 1 ครั้ง
treatments	ผลการรักษา	ผลวินิจฉัย การรักษา และเวลาที่บันทึก ของคิวที่ตรวจแล้ว	ผลการรักษาของคิว 1 คิว

2. ทำไมแบ่งเป็น 6 ตาราง

แบ่งตาม "สิ่งที่ระบบต้องจำ" 6 อย่าง ซึ่งเกิดขึ้นและเปลี่ยนแปลงไม่พร้อมกัน: แผนกกับแพทย์แทบไม่เปลี่ยน ผู้ป่วยลงทะเบียนครั้งเดียว ช่องเวลาสร้างไว้ล่วงหน้าก่อนมีคนจอง คิวเกิดใหม่ทุกครั้งที่มีการจอง และผลการรักษาเกิดตอนแพทย์ตรวจเสร็จ

- **schedules แยกจาก appointments** เพราะช่องเวลาต้องมีอยู่ก่อนจะมีคนจอง ระบบจึงดึงช่องที่ว่างมาใส่ Combobox ได้ และเมื่อคิวถูกยกเลิก ช่องเวลายังอยู่ให้จองใหม่ได้
- **departments แยกจาก doctors** เพราะแผนกหนึ่งมีแพทย์หลายคน และใช้เป็นตัวกรองตอนจอง (เลือกแผนก → แพทย์)
- **patients แยกจาก appointments** เพราะผู้ป่วย 1 คนจองได้หลายครั้ง และต้องลงทะเบียนไว้ได้แม้ยังไม่จอง
- **treatments แยกจาก appointments** เพราะผลการรักษามีเฉพาะคิวที่ตรวจแล้ว และบันทึกตอนแพทย์ตรวจเสร็จ ไม่ใช่ตอนจอง คิวที่ยังรอตรวจหรือยกเลิกจึงไม่มีแถวใน treatments (ข้อมูลตัวอย่างมีคิว 30 คิว ผลการรักษา 15 แถว)

ถ้าเก็บทุกอย่างไว้ในตารางเดียว

ตัวอย่างตาราง "คิว" ตารางเดียว (ข้อมูลสมมติ):

คิว	ชื่อผู้ป่วย	เบอร์โทร	วันเกิด	แพทย์	แผนก	วันที่	เวลา	สถานะ
1	สมหญิง ใจดี	0812345678	1990-05-01	นพ.ประวิทย์ ศรีสุข	อายุรกรรม	2026-10-23	09:00	จองแล้ว
2	สมหญิง ใจดี	0812345678	1990-05-01	ทพ.วรรณา แก้วงาม	ทันตกรรม	2026-10-30	10:00	จองแล้ว
3	มานะ รักเรียน	0898765432	1985-11-20	นพ.ประวิทย์ ศรีสุข	อายุรกรรม	2026-10-23	09:30	ตรวจแล้ว

- **ข้อมูลซ้ำ:** ชื่อ เบอร์ วันเกิดของสมหญิงถูกเก็บ 2 รอบ และ "นพ.ประวิทย์ — อายุรกรรม" ก็ซ้ำ 2 รอบ ยิ่งจองมากยิ่งซ้ำมาก
- **แก้ไม่ครบ (update anomaly):** สมหญิงเปลี่ยนเบอร์ ต้องแก้ทุกแถว ถ้าพลาดแถวเดียว ระบบจะมีเบอร์ของเธอ 2 เบอร์
- **เพิ่มไม่ได้ (insert anomaly):** ลงทะเบียนผู้ป่วยที่ยังไม่จอง เพิ่มแพทย์ใหม่ หรือเปิดช่องเวลาว่างไม่ได้ เพราะทุกแถวต้องเป็นคิว
- **ลบแล้วหาย (delete anomaly):** ถ้าลบแถวที่ 3 ข้อมูลของมานะ (ชื่อ เบอร์ วันเกิด) หายไปทั้งหมด

เมื่อแยกเป็น 6 ตาราง ข้อมูลของสมหญิงเก็บที่เดียวใน patients ส่วนคิวเก็บแค่ patient_id ชี้กลับไป แก้เบอร์ครั้งเดียวก็ถูกทุกคิว

3. ทำไมอยู่ในรูป 3NF

ทุกตารางผ่าน 1NF, 2NF และ 3NF: ทุกฟิลด์ที่ไม่ใช่คีย์ ขึ้นกับคีย์หลักของตารางตัวเองโดยตรงเท่านั้น

ขั้น	เงื่อนไข	ระบบนี้ผ่านเพราะ
1NF	ทุกช่องเก็บค่าเดียว ไม่มีกลุ่มข้อมูลซ้ำ และมีคีย์หลัก	ช่องเวลา 1 ช่อง = 1 แถว ไม่เก็บ "09:00, 09:30" รวมในช่องเดียว และทุกตารางมีคีย์หลัก (_id)
2NF	ผ่าน 1NF และไม่มีฟิลด์ที่ขึ้นกับคีย์หลักแค่บางส่วน	ทุกตารางใช้คีย์หลักฟิลด์เดียว จึงไม่มีทางขึ้นกับ "บางส่วน" ของคีย์
3NF	ผ่าน 2NF และไม่มีฟิลด์ที่ขึ้นกับฟิลด์ธรรมดาอื่น (transitive dependency)	ตัดฟิลด์ที่หาได้จากตารางอื่นออกหมด ตามรายการด้านล่าง

จุดที่ตั้งใจออกแบบให้ผ่าน 3NF:

- **doctors เก็บ dept_id ไม่เก็บ dept_name** ถ้าเก็บ จะเกิด doctor_id → dept_id → dept_name คือชื่อแผนกขึ้นกับ dept_id ไม่ใช่ doctor_id
- **appointments เก็บแค่ schedule_id ไม่เก็บ doctor_id, work_date, slot_time** เพราะรู้ได้จาก schedule_id อยู่แล้ว ถ้าเก็บซ้ำอาจเกิดกรณีคิวบอกว่าเป็นหมอ A แต่ช่องเวลาบอกว่าเป็นหมอ B เวลาแสดงผลจึงใช้ JOIN
- **treatments เก็บแค่ appt_id ไม่เก็บ patient_id** เพราะรู้ได้จาก appt_id → appointments.patient_id อยู่แล้ว ถ้าเก็บซ้ำจะเกิด treatment_id → appt_id → patient_id และอาจเกิดกรณีผลการรักษาบอกว่าเป็นผู้ป่วย A แต่คิวเป็นของผู้ป่วย B ประวัติการรักษาจึงใช้ JOIN (รายละเอียดในหัวข้อ 4)
- **patients ไม่เก็บอายุ** เพราะคำนวณจาก birth_date ได้ และอายุเปลี่ยนทุกปี
- **phone และ treatments.appt_id ที่เป็น UNIQUE** เป็นคีย์คู่แข่ง (candidate key) ซึ่งไม่ขัด 3NF เช่นเดียวกับชุด (doctor_id, work_date, slot_time) ใน schedules

4. Data Dictionary

ใช้ชนิดข้อมูลของ SQLite แค่ 2 ชนิด คือ INTEGER กับ TEXT ข้อบังคับทุกข้อในตารางด้านล่างทดลองรันจริงบนเครื่องที่ใช้ทำโปรเจกต์ (Python 3.15, SQLite 3.53.4) เมื่อ 23 ต.ค. 2026 แล้ว ลองใส่ข้อมูลผิด 25 แบบ ถูกปฏิเสธครบทุกแบบ

- SQLite ไม่บังคับความยาวของ VARCHAR(n) จึงเขียน TEXT ตรงๆ
- SQLite ไม่มีชนิด DATE แยก วันที่และเวลาจึงเก็บเป็น TEXT แบบ YYYY-MM-DD และ HH:MM ซึ่งใช้กับฟังก์ชัน date() ได้ และเรียงตามตัวอักษรแล้วตรงกับลำดับเวลา
- CHECK เบอร์โทรและเวลาใช้ GLOB ของ SQLite เช่น เบอร์ต้องเป็น '0' ตามด้วยตัวเลขอีก 9 หลัก ส่วน CHECK วันที่ใช้ date(x) IS x: ถ้าเป็นวันที่จริง date() คืนค่าเดิม ถ้าผิด (เช่น 2026-02-30) ได้ค่าอื่นหรือ NULL จึงถูกปฏิเสธ

departments (แผนก)

ฟิลด์	ชนิด	คีย์	ข้อบังคับ	คำอธิบาย
dept_id	INTEGER	PK	AUTOINCREMENT	รหัสแผนก ระบบรันเลขให้
dept_name	TEXT	—	NOT NULL, UNIQUE	ชื่อแผนก เช่น อายุรกรรม ห้ามซ้ำ

doctors (แพทย์)

ฟิลด์	ชนิด	คีย์	ข้อบังคับ	คำอธิบาย
doctor_id	INTEGER	PK	AUTOINCREMENT	รหัสแพทย์
doctor_name	TEXT	—	NOT NULL	ชื่อ-นามสกุลแพทย์ เช่น นพ.ประวิทย์ ศรีสุข
dept_id	INTEGER	FK → departments.dept_id	NOT NULL	แผนกที่สังกัด (แพทย์ 1 คน อยู่ 1 แผนก)

patients (ผู้ป่วย)

ฟิลด์	ชนิด	คีย์	ข้อบังคับ	คำอธิบาย
patient_id	INTEGER	PK	AUTOINCREMENT	รหัสผู้ป่วย
patient_name	TEXT	—	NOT NULL	ชื่อ-นามสกุลผู้ป่วย
phone	TEXT	—	NOT NULL, UNIQUE, CHECK ตัวเลข 10 หลัก ขึ้นต้นด้วย 0	เบอร์โทร ใช้ค้นหาผู้ป่วยตอนจอง เก็บเป็น TEXT เพราะถ้าเป็น INTEGER เลข 0 ตัวหน้าจะหาย
birth_date	TEXT	—	NOT NULL, CHECK เป็นวันที่จริงแบบ YYYY-MM-DD	วันเกิด (ค.ศ.) ห้ามเป็นวันในอนาคต ส่วนนี้โปรแกรมเช็กเพราะ CHECK ใช้ date('now') ไม่ได้ (ช่วยจับกรณีพิมพ์ปี พ.ศ. ผิดได้ด้วย)

schedules (ช่องเวลาออกตรวจ)

ฟิลด์	ชนิด	คีย์	ข้อบังคับ	คำอธิบาย
schedule_id	INTEGER	PK	AUTOINCREMENT	รหัสช่องเวลา
doctor_id	INTEGER	FK → doctors.doctor_id	NOT NULL	แพทย์เจ้าของช่องเวลา
work_date	TEXT	—	NOT NULL, CHECK เป็นวันที่จริงแบบ YYYY-MM-DD	วันที่ออกตรวจ (ค.ศ.)
slot_time	TEXT	—	NOT NULL, CHECK รูปแบบ HH:MM (ชั่วโมง 00–23) และนาทีเป็น 00 หรือ 30	เวลาเริ่มของช่อง ช่องละ 30 นาที เช่น 09:00 คือ 09:00–09:30

ข้อบังคับระดับตาราง: **UNIQUE (doctor_id, work_date, slot_time)** แพทย์คนเดียวกันห้ามมีวันและเวลาซ้ำ (แพทย์คนละคน เวลาเดียวกันได้)

การสร้างช่องเวลาล่วงหน้า 30 วัน: แพทย์ทุกคนออกตรวจทุกวันรวมเสาร์-อาทิตย์ ช่วง 09:00–12:00 และ 13:00–16:00 ช่องละ 30 นาที วันละ 12 ช่อง (เวลาเริ่ม 09:00 ถึง 11:30 และ 13:00 ถึง 15:30) พักเที่ยง 12:00–13:00 ช่องเหล่านี้ไม่ได้พิมพ์ INSERT ทีละแถวใน clinic_db.sql แต่ create_db.py สร้างให้หลังใส่ข้อมูลตัวอย่าง ตั้งแต่วันที่รันถึงอีก DAYS_AHEAD = 30 วัน (ค่าคงที่ แก้ตัวเลขเดียวถ้าจะเปิดจองล่วงหน้ามากหรือน้อยกว่านี้):

```
DAYS_AHEAD = 30
SLOT_TIMES = ['09:00', '09:30', '10:00', '10:30', '11:00', '11:30',
              '13:00', '13:30', '14:00', '14:30', '15:00', '15:30']

new_slots = []
for day in range(DAYS_AHEAD + 1):
    work_date = (date.today() + timedelta(days=day)).isoformat()
    for doctor_id in doctor_ids:
        for slot_time in SLOT_TIMES:
            new_slots.append((doctor_id, work_date, slot_time))

cursor.executemany('''INSERT OR IGNORE INTO schedules (doctor_id, work_date, slot_time)
                      VALUES (?, ?, ?)''', new_slots)
```

- **ลูป 3 ชั้น** วัน (0 = วันนี้ ถึง 30) × แพทย์ทุกคน × 12 เวลา ได้รายการ (doctor_id, work_date, slot_time) แล้วส่งให้ executemany ใส่ทีเดียว ค่าทุกตัวส่งผ่าน ?
- **INSERT OR IGNORE** ข้อมูลตัวอย่างมีช่องเวลาของวันนี้ถึงอีก 2 วันอยู่แล้ว 31 ช่อง (คิวตัวอย่างของช่วงนี้ใช้อยู่ 14 ช่อง) ช่องเหล่านี้ชน UNIQUE (doctor_id, work_date, slot_time) จึงถูกข้ามแทนที่จะเกิด error ช่องตัวอย่างคง schedule_id 1–50 เดิม คิวตัวอย่างจึงยังอ้างช่องเวลาเดิมถูกต้อง
- **schedule_id ของช่องใหม่** ได้เลข 55–3770 เลขข้ามไป 31 เลขเท่ากับจำนวนแถวที่ถูกข้าม เพราะ AUTOINCREMENT นับเลขให้แถวที่ถูกข้ามด้วย ไม่กระทบการทำงาน
- **ช่องของวันที่ผ่านมาแล้ว** 19 ช่องมาจากข้อมูลตัวอย่าง (คิวประวัติใช้อยู่ 16 ช่อง) ยังอยู่ครบ

ผลจากการรันจริง python create_db.py เมื่อ 31 ต.ค. 2026:

รายการ	จำนวนแถว
ช่องที่เตรียมในลูป (31 วัน × แพทย์ 10 คน × 12 ช่อง)	3720
ชนกับช่องตัวอย่าง จึงถูกข้าม (OR IGNORE)	31
เพิ่มใหม่	3689
schedules ทั้งหมด (ช่องตัวอย่าง 50 + เพิ่มใหม่ 3689)	3739
ช่องตั้งแต่วันนี้ (2026-10-31 ถึง 2026-11-30)	3720
ช่องของวันที่ผ่านมาแล้ว (2026-04-14 ถึง 2026-10-30)	19

ตารางอื่นมีจำนวนแถวเท่าเดิม: departments 5, doctors 10, patients 15, appointments 30, treatments 15

appointments (การจองคิว)

ฟิลด์	ชนิด	คีย์	ข้อบังคับ	คำอธิบาย
appt_id	INTEGER	PK	AUTOINCREMENT	รหัสคิว
patient_id	INTEGER	FK → patients.patient_id	NOT NULL	ผู้ป่วยที่จอง
schedule_id	INTEGER	FK → schedules.schedule_id	NOT NULL	ช่องเวลาที่จอง (บอกแพทย์ วันเวลา)
symptom	TEXT	—	ไม่บังคับ (ว่างได้ เก็บเป็น NULL)	อาการเบื้องต้นที่เจ้าหน้าที่จดตอนจอง ไม่กรอกก็ได้
status	TEXT	—	NOT NULL, DEFAULT 'จองแล้ว', CHECK ต้องเป็น จองแล้ว / ตรวจแล้ว / ยกเลิก	สถานะคิว
booked_at	TEXT	—	NOT NULL, DEFAULT วันเวลาปัจจุบันของเครื่อง	วันเวลาที่บันทึกการจอง แบบ YYYY-MM-DD HH:MM:SS ระบบใส่ให้เอง

ดัชนีพิเศษ: **UNIQUE INDEX บน schedule_id เฉพาะแถวที่ status ไม่ใช่ 'ยกเลิก'** (partial index) ช่องเวลาหนึ่งมีคิวที่ยังไม่ยกเลิกได้แค่ 1 แถว แต่มีแถว "ยกเลิก" ได้หลายแถว ทดสอบแล้ว: จองซ้ำช่องเดิมถูกปฏิเสธ ยกเลิกแล้วจองใหม่ได้

treatments (ผลการรักษา)

ฟิลด์	ชนิด	คีย์	ข้อบังคับ	คำอธิบาย
treatment_id	INTEGER	PK	AUTOINCREMENT	รหัสผลการรักษา
appt_id	INTEGER	FK → appointments.appt_id	NOT NULL, UNIQUE	คิวที่บันทึกผล (1 คิวมีผลการรักษาได้ไม่เกิน 1 แถว)
diagnosis	TEXT	—	NOT NULL	ผลวินิจฉัย บังคับกรอก เช่น คออักเสบจากไวรัส
treatment	TEXT	—	ไม่บังคับ (ว่างได้ เก็บเป็น NULL)	การรักษาหรือยาที่ให้
recorded_at	TEXT	—	NOT NULL, DEFAULT วันเวลาปัจจุบันของเครื่อง	วันเวลาที่บันทึกผล แบบ YYYY-MM-DD HH:MM:SS ระบบใส่ให้เอง

เหตุผลตาม 3NF ของตารางนี้:

- **ทำไมแยกเป็นตารางของตัวเอง** diagnosis, treatment และ recorded_at เป็นข้อมูลของการตรวจ 1 ครั้ง ซึ่งขึ้นกับ appt_id อย่างเดียว ถ้ารวมไว้ใน appointments ก็ยังขึ้นกับคีย์เดียวกัน แต่คิวที่ยังรอตรวจหรือยกเลิกจะมี 3 ช่องนี้ว่างเป็น NULL (ข้อมูลตัวอย่าง 15 จาก 30 คิว) และบังคับ diagnosis เป็น NOT NULL ไม่ได้ เมื่อแยกตาราง diagnosis จึงบังคับกรอกได้ และกฎ 1 คิวต่อ 0..1 ผลการรักษาบังคับด้วย UNIQUE บน appt_id
- **ทำไมไม่เก็บ patient_id ซ้ำ** ผู้ป่วยหาได้จาก appt_id → appointments.patient_id ถ้าเก็บ patient_id ใน treatments ด้วย จะเป็น transitive dependency (treatment_id → appt_id → patient_id) ซึ่งผิด 3NF และเปิดโอกาสให้ข้อมูลขัดกัน ประวัติการรักษาในแท็บผู้ป่วยจึงดึงด้วย JOIN treatments → appointments → schedules → doctors → departments แล้วกรอง WHERE appointments.patient_id = ?

5. กฎของระบบ: ป้องกันไว้ที่ไหน

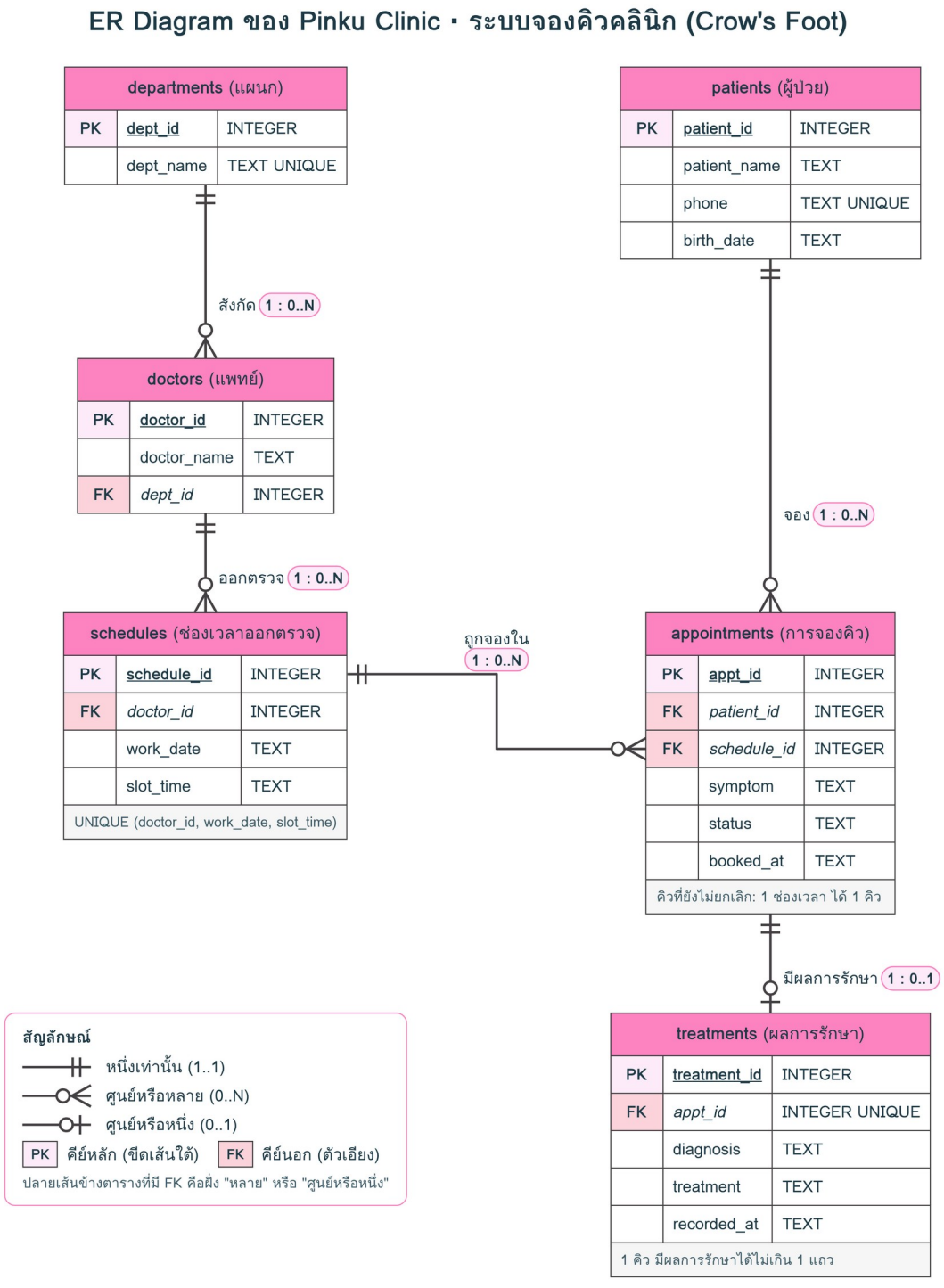
กฎที่ฐานข้อมูลบังคับได้เอง ใส่ไว้ในฐานข้อมูล จึงกันได้แม้มีคนแก้ข้อมูลตรงๆ ผ่าน DB Browser ส่วนกฎที่ต้องรู้ "วันนี้" หรือต้องดูหลายตาราง ให้โปรแกรมเช็กก่อนบันทึก

กฎ	ฐานข้อมูลกันไว้ด้วย	โปรแกรมเช็กเพิ่ม
เบอร์โทรห้ามซ้ำ และต้อง 10 หลัก ขึ้นต้นด้วย 0	UNIQUE + CHECK บน phone	เช็กก่อนบันทึก แจ้งเตือนภาษาไทย
วันเกิดต้องเป็นวันที่จริง และห้ามเป็นวันในอนาคต	CHECK date(birth_date) IS birth_date	เช็กว่าไม่เกินวันนี้ก่อนบันทึก (CHECK ใช้ date('now') ไม่ได้)
แพทย์ห้ามมีช่องเวลาซ้ำวันและเวลาเดียวกัน	UNIQUE (doctor_id, work_date, slot_time)	—
ช่องเวลาหนึ่งมีคิวที่ยังไม่ยกเลิกได้ 1 คิว	UNIQUE แบบมีเงื่อนไข (partial index) บน appointments.schedule_id ไม่นับคิวที่ยกเลิก	Combobox แสดงเฉพาะช่องที่ว่าง และเช็กซ้ำตอนกดบันทึก

กฎ	ฐานข้อมูลกันไว้ด้วย	โปรแกรมเช็กเพิ่ม
ผู้ป่วยห้ามมีคิวที่ยังไม่ยกเลิกซ้อนกันในวันและเวลาเดียวกัน	— ต้อง JOIN กับ schedules จึงทำเป็น UNIQUE ไม่ได้	SELECT หาคิวของผู้ป่วยคนนี้ในวันและเวลาเดียวกันก่อน INSERT
จองได้เฉพาะช่องที่ยังไม่ถึงเวลา (วันพรุ่งนี้เป็นต้นไปทุกช่อง ส่วนวันนี้เฉพาะช่องที่ยังไม่ถึงเวลาปัจจุบัน)	— SQLite ไม่ยอมให้ใช้ date('now') ใน CHECK เพราะค่าเปลี่ยนทุกวัน (ทดสอบแล้ว ได้ error)	Combobox ดึงเฉพาะช่องของวันพรุ่งนี้เป็นต้นไป หรือช่องของวันนี้ที่ slot_time > เวลาปัจจุบัน (วันนี้ไม่มีช่องเหลือ = ไม่แสดงวันนี้) และเช็กซ้ำตอนกดบันทึก
ยกเลิกคิวไม่ลบแถว	—	ใช้ UPDATE status = 'ยกเลิก' ไม่ใช้ DELETE
คิวที่ตรวจแล้วหรือยกเลิกแล้วเปลี่ยนสถานะซ้ำไม่ได้	—	UPDATE มีเงื่อนไข AND status = 'จองแล้ว' และเช็กสถานะก่อนถามยืนยัน
status มีได้ 3 ค่าเท่านั้น	CHECK status IN ('จองแล้ว','ตรวจแล้ว','ยกเลิก')	ให้เลือกจากปุ่ม ไม่ให้พิมพ์เอง
ลบข้อมูลที่ยังถูกอ้างอิงไม่ได้ เช่น ผู้ป่วยที่มีคิว	FOREIGN KEY ทุกเส้น	สั่ง PRAGMA foreign_keys = ON ทุกครั้งที่เชื่อมต่อ (ไม่สั่ง FK จะไม่ทำงาน)
ไม่ต่อสตริงเป็นคำสั่ง SQL	—	ทุกคำสั่งใช้ ? แทนค่า
บันทึกว่าตรวจแล้วกับผลการรักษาต้องสำเร็จพร้อมกัน (สำเร็จทั้งคู่หรือไม่บันทึกเลย)	— เป็นคำสั่งของ 2 ตาราง	กดยืนยัน → ฟอร์มผลวินิจฉัย → UPDATE appointments กับ INSERT treatments ใน transaction เดียว (database.execute_transaction) ถ้าคำสั่งใดผิดหรือ UPDATE ไม่เจอคิวที่ยังเป็น 'จองแล้ว' จะ rollback ทั้งหมด
ทุกคิวที่ตรวจแล้วต้องมีผลการรักษา	— CHECK ดูข้ามตารางไม่ได้	สถานะ 'ตรวจแล้ว' เกิดได้ทางเดียวคือ transaction ในข้อบน และ create_db.py ตรวจซ้ำว่าคิวที่ตรวจแล้วแต่ไม่มีผลการรักษาเป็น 0 คิว
1 คิวมีผลการรักษาได้ไม่เกิน 1 แถว	UNIQUE + FOREIGN KEY บน treatments.appt_id	ถ้าบันทึกซ้ำ แจ้งเตือนเป็นภาษาไทย
ผลวินิจฉัยบังคับกรอก การรักษาไม่บังคับ	NOT NULL บน diagnosis	ฟอร์มเตือนถ้าผลวินิจฉัยว่าง ช่องการรักษาที่เว้นว่างเก็บเป็น NULL
บันทึกว่าตรวจแล้วได้เฉพาะคิวของวันนี้หรือวันที่ผ่านมา	—	เช็กวันที่ของคิวก่อนเปิดฟอร์มผลวินิจฉัย

6. ER Diagram และ cardinality

มี 5 ความสัมพันธ์: 4 เส้นเป็นแบบ 1 ต่อหลาย (1:N) และ appointments กับ treatments เป็นแบบ 1 ต่อ 0..1 ส่วน patients กับ schedules เป็นหลายต่อหลาย (M:N) ที่เชื่อมกันผ่าน appointments



ER Diagram แบบ Crow's Foot · 6 ตาราง 5 ความสัมพันธ์ (ไฟล์ docs/er_diagram.png)

ความสัมพันธ์	cardinality	อ่านเป็นประโยค
departments → doctors (สังกัด)	1 : 0..N	แผนก 1 แผนกมีแพทย์ได้หลายคน (หรือยังไม่มีเลยก็ได้) และแพทย์ 1 คนสังกัดได้แผนกเดียว
doctors → schedules (ออกตรวจ)	1 : 0..N	แพทย์ 1 คนมีช่องเวลาออกตรวจได้หลายช่อง และช่องเวลา 1 ช่องเป็นของแพทย์คนเดียว
patients → appointments (จอง)	1 : 0..N	ผู้ป่วย 1 คนจองคิวได้หลายครั้ง (หรือลงทะเบียนไว้แต่ยังไม่จองก็ได้) และคิว 1 คิวเป็นของผู้ป่วยคนเดียว
schedules → appointments (ถูกจองใน)	1 : 0..N	ช่องเวลา 1 ช่องมีประวัติคิวได้หลายแถว แต่คิวที่ยังไม่ยกเลิกมีได้แค่ 1 คิว และคิว 1 คิวอยู่ในช่องเวลาเดียว
appointments → treatments (มีผลการรักษา)	1 : 0..1	คิว 1 คิวมีผลการรักษาได้ไม่เกิน 1 แถว (คิวที่ยังรอตรวจหรือยกเลิกไม่มี) และผลการรักษา 1 แถวเป็นของคิวเดียว

เส้น schedules → appointments เขียนเป็น 0..N ไม่ใช่ 0..1 เพราะคิวที่ยกเลิกยังเก็บไว้ ช่องเวลาเดียวจึงมีได้หลายแถว ส่วนกฎ "คิวที่ยังไม่ยกเลิกได้ 1 คิว" บังคับด้วย partial index ในหัวข้อ 4

สัญลักษณ์ || ฝั่ง 1 แปลว่าแถวฝั่งตรงข้ามต้องอ้างถึงแถวฝั่ง 1 ได้พอดี 1 แถวเสมอ ห้ามว่าง FK ทั้ง 5 ตัวจึงเป็น NOT NULL: doctors.dept_id, schedules.doctor_id, appointments.patient_id, appointments.schedule_id และ treatments.appt_id ส่วนสัญลักษณ์ o| ฝั่ง treatments แปลว่าคิว 1 คิวมีผลการรักษาได้ 0 หรือ 1 แถว ซึ่งบังคับด้วย UNIQUE บน treatments.appt_id

7. SQL สรุปตัวเลขวันนี้ (แถบหัวโปรแกรม)

แถบหัวโปรแกรมแสดงสรุปคิววันนี้ 4 ตัวเลข ได้จาก SQL คำสั่งเดียวใน database.get_today_summary() ซึ่งใช้ COUNT(*) นับคิวทั้งหมด และ SUM(CASE ...) นับแยกตามสถานะ

```
SELECT date('now', 'localtime'),
       COUNT(*),
       IFNULL(SUM(CASE WHEN a.status = 'ตรวจแล้ว' THEN 1 ELSE 0 END), 0),
       IFNULL(SUM(CASE WHEN a.status = 'จองแล้ว' THEN 1 ELSE 0 END), 0),
       IFNULL(SUM(CASE WHEN a.status = 'ยกเลิก' THEN 1 ELSE 0 END), 0)
FROM appointments a
JOIN schedules s ON a.schedule_id = s.schedule_id
WHERE s.work_date = date('now', 'localtime')
```

ผลจากการรันจริงบนเครื่อง หลัง python create_db.py เมื่อ 23 ต.ค. 2026 ตรงกับตัวเลขบนแถบหัว:

วันที่	คิวทั้งหมด	ตรวจแล้ว	รอตรวจ (จองแล้ว)	ยกเลิก
2026-10-23	7	0	6	1

- **ต้อง JOIN schedules** เพราะวันที่ของคิวอยู่ที่ schedules.work_date (appointments ไม่เก็บวันซ้ำตาม 3NF)
- **COUNT(*)** นับทุกคิวของวันนี้ รวมคิวที่ยกเลิก ส่วน **SUM(CASE WHEN ... THEN 1 ELSE 0 END)** นับเฉพาะแถวที่ตรงสถานะ จึงได้ 4 ตัวเลขในแถวเดียว (7 = 0 + 6 + 1)
- **IFNULL(..., 0)** วันที่ไม่มีคิวเลย SUM จะได้ NULL จึงแปลงเป็น 0

- **ตัวเลขอัปเดตเมื่อ** จองคิว ยกเลิกคิว บันทึกว่าตรวจแล้ว และตอนสลับแท็บ (เรียก header_bar.refresh())
- ข้อมูลตัวอย่างใช้วันที่นับจากวันที่รัน create_db.py (เช่น date('now','localtime','+1 day')) หลังรีเซ็ตจึงได้ตัวเลขชุดนี้ แต่วันที่เปลี่ยนตามวันที่รัน

8. SQL สรุปตัวเลขรายวัน (หน้าเว็บ app.py)

หน้าเว็บ app.py แยกตารางวันละ 1 ตาราง และมีกล่องสรุป 4 กล่องต่อวัน (คิวทั้งหมด / ตรวจแล้ว / รอตรวจ / ยกเลิก) ตัวเลขทุกวันได้จาก SQL คำสั่งเดียวที่ GROUP BY วันที่กับสถานะ

```
SELECT s.work_date, a.status, COUNT(*)
FROM appointments a
JOIN schedules s ON a.schedule_id = s.schedule_id
GROUP BY s.work_date, a.status
ORDER BY CASE WHEN s.work_date >= ? THEN 0 ELSE 1 END,
         CASE WHEN s.work_date >= ? THEN s.work_date END,
         s.work_date DESC
```

ค่าที่แทน ? ทั้ง 2 ตัวคือวันนี้แบบ YYYY-MM-DD ผลจากการรันจริงกับ clinic.db ที่สร้างเมื่อ 31 ต.ค. 2026 ได้ 18 แถว ครอบคลุม 15 วันที่มีคิว 5 แถวแรกคือวันนี้และวันข้างหน้า:

work_date	status	COUNT(*)
2026-10-31	จองแล้ว	6
2026-10-31	ยกเลิก	1
2026-11-01	จองแล้ว	3
2026-11-01	ยกเลิก	1
2026-11-02	จองแล้ว	3

แถวถัดไปเป็นวันที่ผ่านมาแล้วเรียงจากใหม่ไปเก่า เริ่มที่ 2026-10-30 ตรวจแล้ว 3 จนถึง 2026-04-14 ตรวจแล้ว 1

- **GROUP BY s.work_date, a.status** รวมคิวที่วันและสถานะเดียวกันเป็นกลุ่มเดียว แล้ว COUNT(*) นับจำนวนคิวในกลุ่ม ตัวเลขจึงนับด้วย SQL ไม่ได้วนนับใน Python
- **ต้อง JOIN schedules** เพราะวันที่ของคิวอยู่ที่ schedules.work_date เหมือนหัวข้อ 7
- **ORDER BY 3 ชั้น** ชั้นแรกแบ่งกลุ่ม 0 = วันนี้และวันข้างหน้า กลุ่ม 1 = วันที่ผ่านมาแล้ว ชั้นที่สองเรียงกลุ่ม 0 จากใกล้ไปไกล (กลุ่ม 1 ได้ NULL เท่ากันหมด) ชั้นที่สามเรียงกลุ่ม 1 จากใหม่ไปเก่า หน้าเว็บจึงมีวันนี้อยู่บนสุด
- **สถานะที่ไม่มีคิว ไม่มีแถว** เช่น วันนี้ไม่มีแถว "ตรวจแล้ว" app.py จึงตั้งค่าเริ่มต้นของทั้ง 3 สถานะเป็น 0 ก่อนใส่ตัวเลขจาก SQL และวันที่ไม่มีคิวเลยจะไม่มีตารางบนหน้าเว็บ
- **คิวทั้งหมด = ตรวจแล้ว + รอตรวจ + ยกเลิก** app.py บวกตัวเลข 3 สถานะที่ SQL นับมา วันนี้จึงได้ 7 = 0 + 6 + 1 ตรงกับแถบหัวในหัวข้อ 7
- **ใช้ ? แทนวันนี้** ไม่ต่อวันที่เป็นข้อความในคำสั่ง SQL ส่วนรายการคิวในแต่ละตารางมาจาก SELECT อีกคำสั่งที่ JOIN 5 ตารางแล้วเรียงตามวันและเวลา หน้าเว็บมีแต่ SELECT ไม่แก้ข้อมูล

9. ข้อสรุปที่ยืนยันแล้ว

ยืนยันกับทีมเมื่อ 22 ต.ค. 2026 และใช้ตามนี้ในขั้นที่ 2 เป็นต้นไป

1. **symptom ไม่บังคับกรอก** ว่างได้ เก็บเป็น NULL
2. **birth_date บังคับกรอก** ต้องเป็นวันที่จริงแบบ YYYY-MM-DD (CHECK ในฐานข้อมูล) และห้ามเป็นวันในอนาคต (โปรแกรมเช็ก)
3. **กันคิวซ้ำ 2 ชั้น** UNIQUE แบบมีเงื่อนไข (partial index) ที่ไม่นับคิวที่ยกเลิก + โปรแกรมเช็ก และขั้นที่ 2 มีตัวอย่างยกเลิกคิวแล้วจองช่องเดิมใหม่ได้
4. **กฎผู้ป่วยห้ามมีคิวซ้อน** โปรแกรมเช็กด้วย SELECT ไม่ใช้ TRIGGER
5. **คีย์หลักใช้ AUTOINCREMENT**
6. **FK ทุกตัวเป็น NOT NULL** ตรงกับสัญลักษณ์ || ใน ER Diagram

ปรับเพิ่มให้สอดคล้องกัน: work_date ใช้ CHECK วันที่จริงแบบเดียวกับ birth_date และ slot_time จำกัดชั่วโมงไว้ที่ 00–23

เพิ่มภายหลัง (Edit Request V1): ตาราง treatments เก็บผลการรักษา (appointments 1 ต่อ 0..1 treatments) บันทึกว่าตรวจแล้วพร้อมผลการรักษาใน transaction เดียว และแถบหัวแสดงตัวเลขสรุปคิววันนี้ ตามหัวข้อ 4–7

เพิ่มภายหลัง (Edit Request V3): ช่องเวลาออกตรวจล่วงหน้า 30 วันที่ create_db.py สร้างด้วยลูป + INSERT OR IGNORE (หัวข้อ 4 ตาราง schedules) และ SQL สรุปตัวเลขรายวันของหน้าเว็บ (หัวข้อ 8) โครงสร้าง 6 ตารางและ ER Diagram ไม่เปลี่ยน